EXPERIMENT 10

AIM: Explore and implement Autoencoders for image denoising

Theory:

1. **Dataset Source**

The dataset used for this experiment is the **MNIST Dataset**.

Source: https://www.kaggle.com/datasets/hojjatk/mnist-dataset

2. **Dataset Description**

The MNIST dataset is a widely used benchmark for deep learning in image processing.

Key Characteristics:

- Type: Grayscale image dataset
- Total Samples: 70,000 images
  Training: 60,000
  Testing: 10,000
- Image Size: 28 × 28 pixels
- Channels: 1 (grayscale)
- Pixel Range: 0–255

Features:

- Each image consists of pixel intensity values.
- After preprocessing, the values are normalized to the range [0,1].

Target Variable:

- No explicit target labels are used in this experiment.
- Input: Noisy image
- Output: Original clean image

This makes it an unsupervised learning problem.

3. **Mathematical Formulation of the Algorithm** An Autoencoder consists of two components:

(a) Encoder

Maps input to a lower-dimensional representation:

$$h = f(x) = \sigma(Wx + b)$$

Where:

- x = input image
- h = latent representation
- W, b = weights and bias
- σ = activation function

(b) Decoder

Reconstructs the original input:

$$\hat{x} = g(h) = \sigma(W'h + b')$$

(c) Loss Function

The model minimizes reconstruction error:

$$L = -\sum [x \log(\hat{x}) + (1 - x) \log(1 - \hat{x})]$$

(Binary Cross-Entropy Loss)

(d) Denoising Objective

The noisy input is defined as:

$$x_{noisy} = x + noise$$

The model learns the mapping:

$$f(x_{noisy}) \approx x$$

4. Algorithm Limitations

- Cannot generalize well to completely unseen data patterns
- Performance depends on the type and level of noise

- May produce blurred reconstructed images
- Requires a large amount of training data
- Overfitting may occur with complex architectures
- Not suitable for high-resolution images without deeper networks

5. Methodology / Workflow

Steps:

1. Load the dataset
2. Perform data preprocessing
   - Normalize pixel values
   - Reshape images
3. Add noise to the images
4. Construct the autoencoder model
   - Encoder (Convolution + Pooling)
   - Decoder (Upsampling / Transposed Convolution)
5. Train the model
   - Input: Noisy images
   - Output: Clean images
6. Generate predictions
7. Visualize results

Workflow Representation:

Original Image → Add Noise → Noisy Image → Autoencoder → Latent Representation → Decoder → Reconstructed Image

6. Performance Analysis

Performance is evaluated using reconstruction quality.

Metrics:

1. Loss Function
   - Binary Cross-Entropy Loss
   - Lower value indicates better reconstruction
2. Mean Squared Error

$$MSE = \frac{1}{n}\sum(x - \hat{x})^2$$

3. Visual Comparison

- Noisy image
- Original image

- Denoised output

Interpretation:

- If the reconstructed image is close to the original, the model performs well
- Slight blurring is acceptable
- Higher loss indicates poor reconstruction

7. Hyperparameter Tuning

Important Parameters:

- Epochs: Number of training iterations
- Batch Size: Number of samples per batch
- Learning Rate: Controls weight updates
- Noise Factor: Amount of noise added
- Number of Filters: Determines feature extraction capability

Tuning Process:

- Start with default parameters
- Adjust epochs to improve learning
- Modify the noise factor to test robustness
- Tune architecture (layers and filters)
- Observe loss and output quality

Impact:

- Proper tuning improves reconstruction quality
- Poor tuning leads to distorted or blurred outputs

<u>CODE and OUTPUT:</u>
Step 1: Import Libraries

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, Conv2DTranspose
from tensorflow.keras.models import Model
from tensorflow.keras.datasets import mnist
```

Step 2: Load Dataset (MNIST)

```
(x_train, _), (x_test, _) = mnist.load_data()

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
```

Step 3: Preprocess Data

```
x_train = x_train.astype('float32') / 255.
x_test = x_test.astype('float32') / 255.

x_train = np.reshape(x_train, (-1, 28, 28, 1))
x_test = np.reshape(x_test, (-1, 28, 28, 1))
```

Step 4: Add Noise

```
noise_factor = 0.5

x_train_noisy = x_train + noise_factor * np.random.normal(size=x_train.shape)
x_test_noisy = x_test + noise_factor * np.random.normal(size=x_test.shape)

x_train_noisy = np.clip(x_train_noisy, 0., 1.)
x_test_noisy = np.clip(x_test_noisy, 0., 1.)
```

Step 5: Build Autoencoder Model (Encoder + Decoder)

```
input_img = Input(shape=(28, 28, 1))

# Encoder
x = Conv2D(32, (3,3), activation='relu', padding='same')(input_img)
x = MaxPooling2D((2,2), padding='same')(x)
x = Conv2D(32, (3,3), activation='relu', padding='same')(x)
encoded = MaxPooling2D((2,2), padding='same')(x)

# Decoder
x = Conv2DTranspose(32, (3,3), strides=2, activation='relu', padding='same')(encoded)
x = Conv2DTranspose(32, (3,3), strides=2, activation='relu', padding='same')(x)
decoded = Conv2D(1, (3,3), activation='sigmoid', padding='same')(x)

autoencoder = Model(input_img, decoded)
```

Step 6: Compile Model

```
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')
```

Step 7: Train Model

```
autoencoder.fit(
    x_train_noisy, x_train,
    epochs=10,
    batch_size=128,
    shuffle=True,
    validation_data=(x_test_noisy, x_test)
)
```

```
Epoch 1/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 135s 288ms/step - loss: 0.1200 - val_loss: 0.1126
Epoch 2/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 132s 281ms/step - loss: 0.1111 - val_loss: 0.1078
Epoch 3/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 146s 289ms/step - loss: 0.1077 - val_loss: 0.1056
Epoch 4/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 136s 277ms/step - loss: 0.1056 - val_loss: 0.1041
Epoch 5/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 144s 281ms/step - loss: 0.1043 - val_loss: 0.1029
Epoch 6/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 140s 277ms/step - loss: 0.1032 - val_loss: 0.1019
Epoch 7/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 133s 284ms/step - loss: 0.1024 - val_loss: 0.1011
Epoch 8/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 129s 275ms/step - loss: 0.1016 - val_loss: 0.1007
Epoch 9/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 129s 275ms/step - loss: 0.1011 - val_loss: 0.1000
Epoch 10/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 147s 285ms/step - loss: 0.1006 - val_loss: 0.0995
<keras.src.callbacks.history.History at 0x7fd6eda7b920>
```

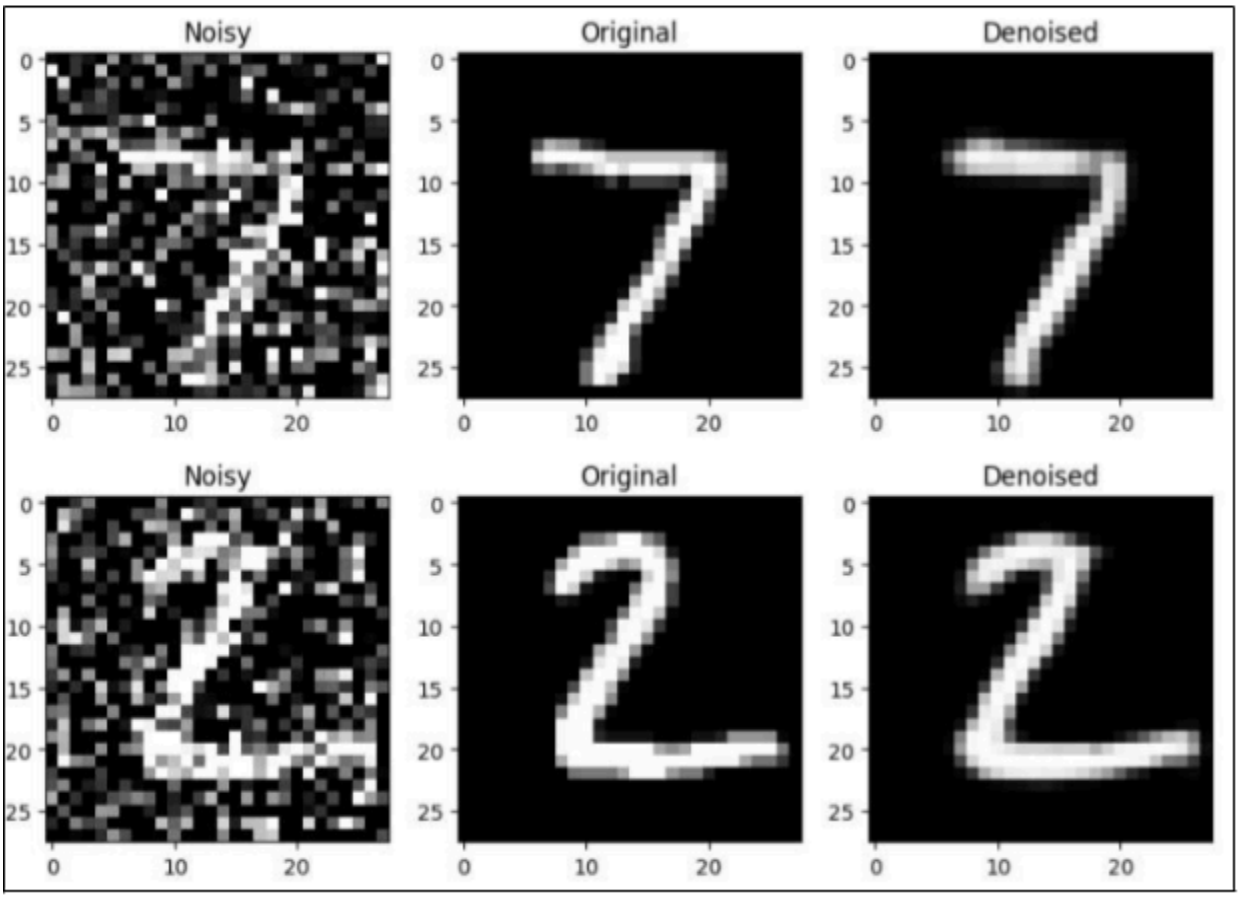
Step 8: Test / Predict

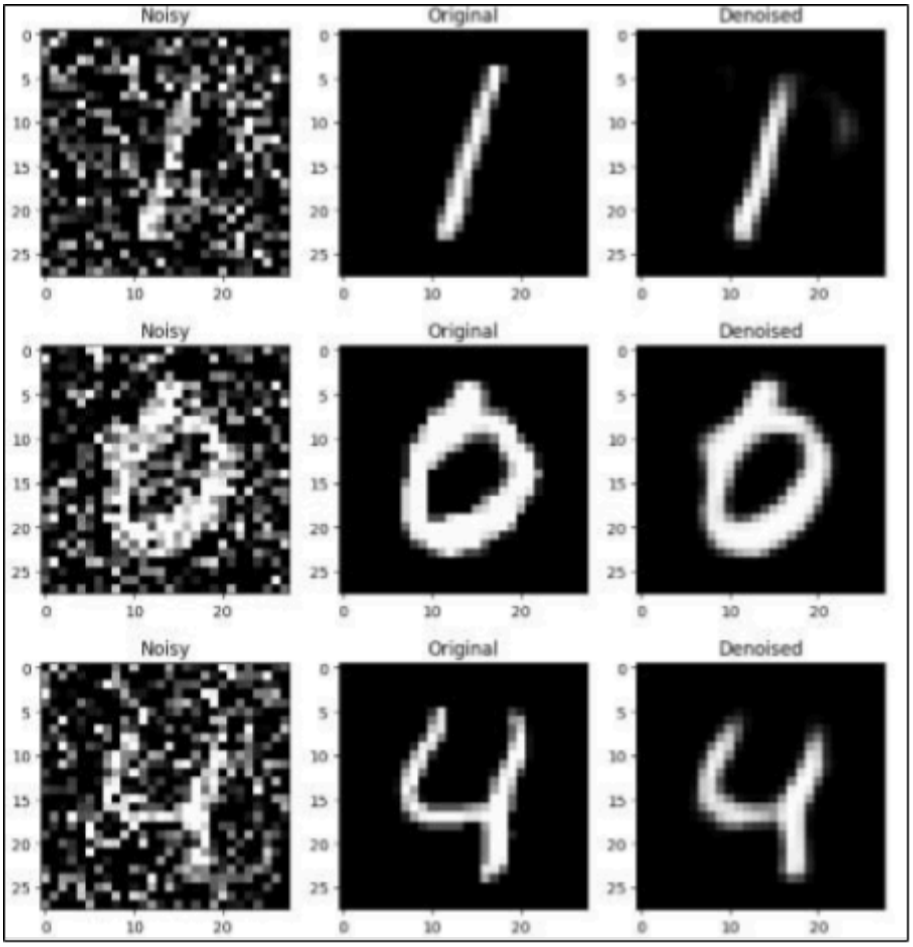
```
decoded_imgs = autoencoder.predict(x_test_noisy)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step
```

Step 9: Display Results

```
n = 5
for i in range(n):
    plt.figure(figsize=(10,4))

    # Noisy
    plt.subplot(1,3,1)
    plt.imshow(x_test_noisy[i].reshape(28,28), cmap='gray')
    plt.title("Noisy")

    # Original
    plt.subplot(1,3,2)
    plt.imshow(x_test[i].reshape(28,28), cmap='gray')
    plt.title("Original")

    # Denoised
    plt.subplot(1,3,3)
    plt.imshow(decoded_imgs[i].reshape(28,28), cmap='gray')
    plt.title("Denoised")

    plt.show()
```





Conclusion

Autoencoders are effective models for image denoising tasks. They learn compact representations of data and reconstruct clean images from noisy inputs. Their performance depends on model architecture, hyperparameter tuning, and dataset quality.